2. Concurrent programming part (Go) [8% of your final mark]

We now ask you to create a Go application that produces a stable solution for the residents and programs matching problem. This time you have to use the recursive **McVitie-Wilson** algorithm.

Your solution must include two versions:

- a regular single-threaded solution (i.e. not using concurrency);
- and a concurrent solution that will use goroutines to create multiple threads.

Basically, the single-threaded solution will be similar to the concurrent one in which all the `go` statements have been removed.

The concurrent strategy to use here is simple: all calls to the `offer` function must be done concurrently, that is using goroutines. However, these multiple concurrent calls to the `offer` function will necessitate to protect some of the data structures with **Mutex** in order to avoid simultaneous competitive modifications. It is important to note that all the calls to `offer` should be done as goroutines, including the ones inside the `evaluate` function.

```
go offer(…
```

We provide you with a Go program that reads the csv files and produces two maps: one for the residents and one for the programs; these maps use the IDs as key. Note that these maps contain pointers to the resident or program instances; this is more convenient when you have to modify the values of some of the fields of the contained types (remember that map access return a copy of the corresponding instance). We suggest you to use the following type definitions:

```
// The Resident data type
type Resident struct {
     residentID int
     firstname string
     lastname string
     rol []string              // resident rank order list
     matchedProgram string     // will be "" for unmatched resident
}
```

```
// The Program data type
type Program struct {
     programID string
     name       string
     nPositions  int       // number of positions available (quota)
     rol []int             // program rank order list
     selectedResidents     // TO ADD: a data structure
                           // for the selected resident IDs
}
```

It is a good idea to send these maps to the `offer` and `evaluate` functions together with the current resident and program IDs. The functions would therefore have the following signatures:

```
func offer(rid int, // resident ID
     residents map[int]*Resident, programs map[string]*Program,
     ???possibly other arguments…)

func evaluate(rid int, pid string, // resident and program IDs
     residents map[int]*Resident, programs map[string]*Program,
     ???possibly other arguments…)
```

At the end of the algorithm, each resident is associated to the program identified by the `matchedProgram` field and the programs have a list of accepted residents as represented by the field `selectedResidents`. The exact type of this list is left to you; a heap using the rank order as the key and the ID as the value would certainly constitute a good choice.

Again, you must submit a text file containing the output of your program (for the `residents.csv` and `programs.csv` files) using the format specified in the problem description to which you also add a last line indicating the runtime of your solution:

```
lastname,firstname,residentID,programID,name
Clown,Marie,773,OBG,Obstetrics and Gynecology
Frederick,Rosalie,517,NRS,Neurosurgery
Jarvis,Zev,226,MMI,Microbiology
Medrano,Indie,126,NRS,Neurosurgery
Olson,Aspyn,403,HEP,Hematological Pathology
Paquet,Camille,913,HEP,Hematological Pathology
Robert,Laurent,616,OBG,Obstetrics and Gynecology
```

```
Tan,Tom,377,XXX,NOT_MATCHED
Tremmo,Emma,828,OBG,Obstetrics and Gynecology
Williams,Salvatore,574,NRS,Neurosurgery

Number of unmatched residents: 1
Number of positions available: 1

Execution time: 1.5013ms
```

IMPORTANT: To better demonstrate the benefit of using of concurrency, we ask you to also use the `residentsLarge.csv` and `programsLarge.csv` files. In fact, the difference in execution time will probably be only significant for these large files. Note that, the execution time does not include the time it takes to read the input file or to display the solution (never write to the console when you measure execution time). Your `main` code should therefore look as follows:

```
…
start := time.Now(); // chrono

var wg sync.WaitGroup

// try to match each resident
for id, _ := range residents {

     …
     go offer(id, residents, programs, …)
     …
}

wg.Wait()

end := time.Now(); // chrono

…
// print solution
…
fmt.Printf("\n\nExecution time: %s", end.Sub(start))
```

Submission

In addition to the source code of your solution, you must also submit three text files showing the output obtained when running your program:

1. with the `residents.csv` and `programs.csv` files (concurrent or single-threaded version).
2. with the `residentsLarge.csv` and `programsLarge.csv` files (single-threaded version).
3. with the `residentsLarge.csv` and `programsLarge.csv` files (concurrent version).

Recall that the concurrent version is the one using goroutines and the single-threaded version is the one for which you removed all the `go` keywords. Obviously, the concurrent version should be a few times faster that the non-concurrent one.